

Suricata IDS Detection of SQL Injection (DVWA)

A. Title and Group Member

Title

SQL Injection Detection

Group Member

B. Objective of the Project

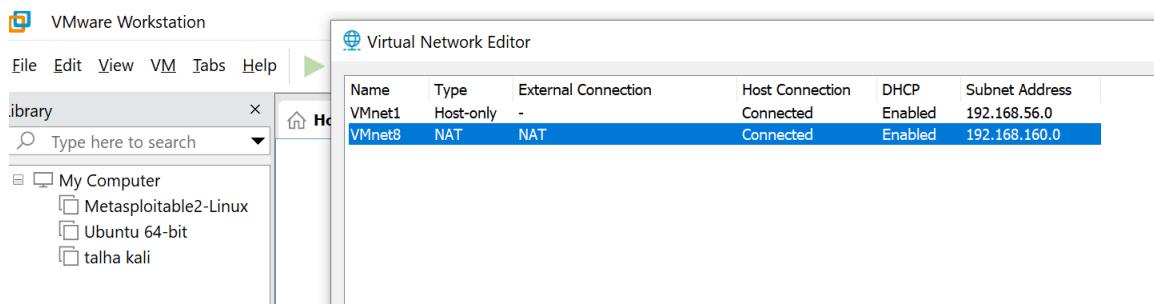
The objective of this project is to demonstrate the detection of a SQL Injection attack using the Suricata Intrusion Detection System (IDS). The attack traffic is generated from an attacker machine (Kali Linux) targeting a vulnerable web application (DVWA) hosted on Metasploitable 2, while an Ubuntu machine running Suricata monitors the network traffic. The selected attack type is SQL Injection (SQLi).

C. Tools Used

- VMware Workstation
- Kali Linux (Attacker)
- Metasploitable 2 (Victim)
- Ubuntu (Suricata IDS)

D. Step-by-Step Procedure

1. Configure VMware virtual machines on the same network.



2. Set ip of all systems

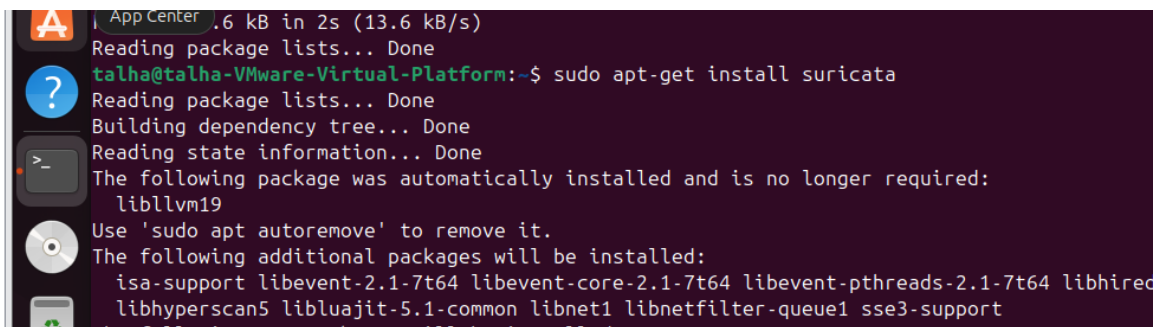
Set all systems Ip on same network (NAT (vmnet-8)customs network)

Kali linux	192.168.160
Ubuntu linux	192.168.160
Metasploitable 2	192.168.160

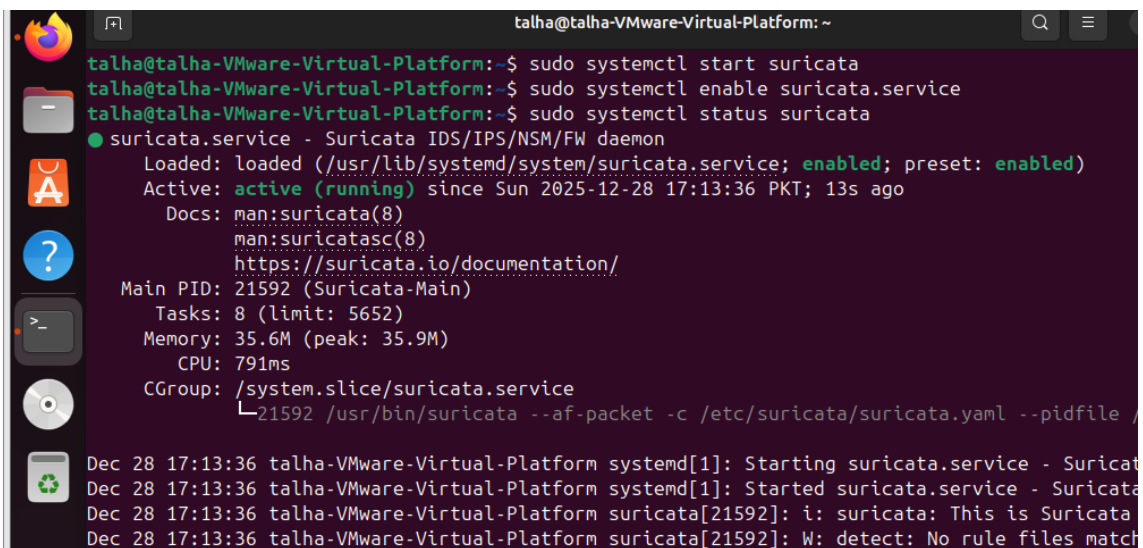
3. Activate Ubuntu and start Suricata

Cmd `sudo apt-get install suricata`

`sudo systemctl status suricata`



```
App Center 1.6 kB in 2s (13.6 kB/s)
Reading package lists... Done
talha@talha-VMware-Virtual-Platform:~$ sudo apt-get install suricata
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following package was automatically installed and is no longer required:
  libllvm19
Use 'sudo apt autoremove' to remove it.
The following additional packages will be installed:
  isa-support libevent-2.1-7t64 libevent-core-2.1-7t64 libevent-pthreads-2.1-7t64 libhiredis
  libhyperscan5 liblua5.1-0 libnet1 libnetfilter-queue1 sse3-support
```



```
talha@talha-VMware-Virtual-Platform:~$ sudo systemctl start suricata
talha@talha-VMware-Virtual-Platform:~$ sudo systemctl enable suricata.service
talha@talha-VMware-Virtual-Platform:~$ sudo systemctl status suricata
● suricata.service - Suricata IDS/IPS/NSM/FW daemon
   Loaded: loaded (/usr/lib/systemd/system/suricata.service; enabled; preset: enabled)
   Active: active (running) since Sun 2025-12-28 17:13:36 PKT; 13s ago
     Docs: man:suricata(8)
           man:suricatasc(8)
           https://suricata.io/documentation/
   Main PID: 21592 (Suricata-Main)
    Tasks: 8 (limit: 5652)
   Memory: 35.6M (peak: 35.9M)
      CPU: 791ms
   CGroup: /system.slice/suricata.service
           └─21592 /usr/bin/suricata --af-packet -c /etc/suricata/suricata.yaml --pidfile /

Dec 28 17:13:36 talha-VMware-Virtual-Platform systemd[1]: Starting suricata.service - Suricata
Dec 28 17:13:36 talha-VMware-Virtual-Platform systemd[1]: Started suricata.service - Suricata
Dec 28 17:13:36 talha-VMware-Virtual-Platform suricata[21592]: i: suricata: This is Suricata
Dec 28 17:13:36 talha-VMware-Virtual-Platform suricata[21592]: W: detect: No rule files match
```

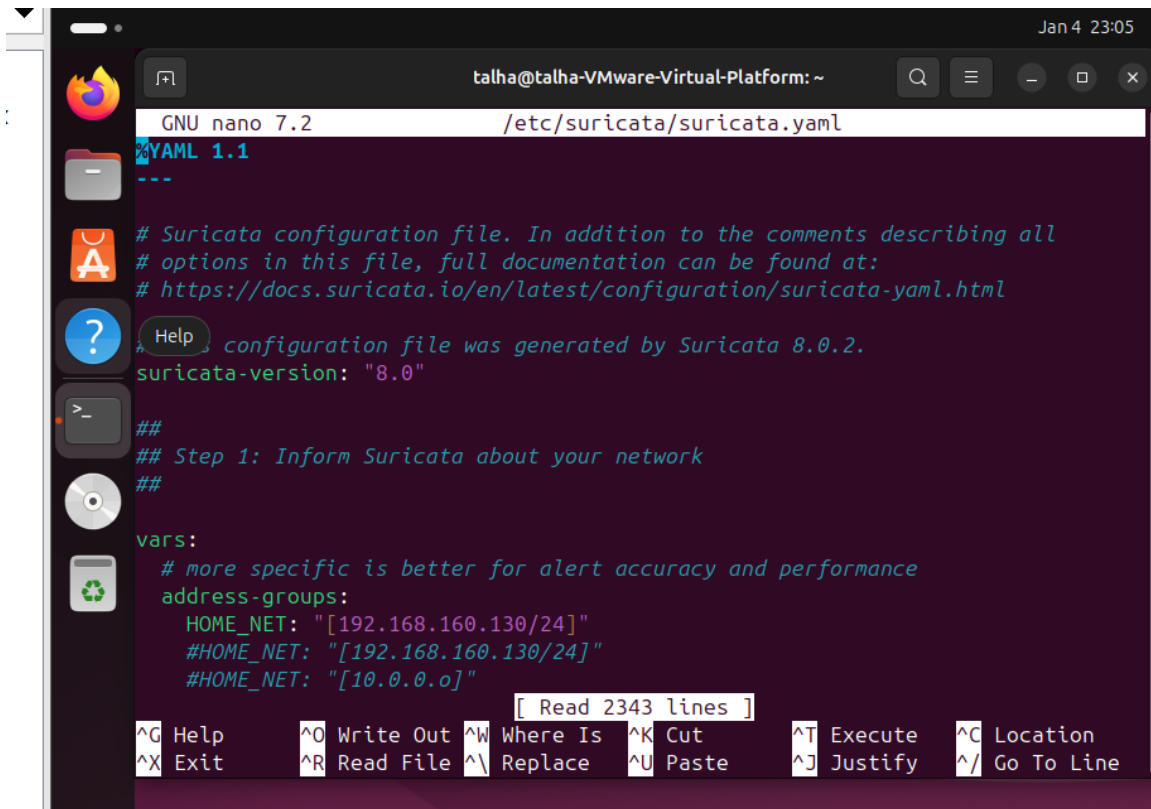
4. Set up Network of Suricata on Metasploitable 2

For detection suricata is set metasploitable 2 to detect Sql injection of kali

So we used metasploiatbale ip

Cmd `sudo nano /etc/suricata/suricata.yaml`

Home_net (Metasploitable IP)



```
talha@talha-VMware-Virtual-Platform: ~
GNU nano 7.2 /etc/suricata/suricata.yaml
%YAML 1.1
---
# Suricata configuration file. In addition to the comments describing all
# options in this file, full documentation can be found at:
# https://docs.suricata.io/en/latest/configuration/suricata-yaml.html
# configuration file was generated by Suricata 8.0.2.
suricata-version: "8.0"

##
## Step 1: Inform Suricata about your network
##

vars:
  # more specific is better for alert accuracy and performance
  address-groups:
    HOME_NET: "[192.168.160.130/24]"
    #HOME_NET: "[192.168.160.130/24]"
    #HOME_NET: "[10.0.0.0]"

[ Read 2343 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute  ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify  ^_ Go To Line
```

5. Local Rules

Setup local rules in suricata for alerts and customs rules for alerts

Make local.rules

Search rules.files in yaml file

See path to local.files to enter

`sudo nano /etc/lib/suricata/rules/suricata.rules`

Make files in which alerts are formed

```

GNU nano 7.2 /etc/suricata/suricata.yaml
# This parameter has no effect, it only exists to prevent
#
hashmode: hash5tuplesorted

##
## Configure Suricata to load Suricata-Update managed rules.
##
default-rule-path: /var/lib/suricata/rules
rule-files:
- suricata.rules
- local.rules

##
## Auxiliary configuration files.
##
classification-file: /etc/suricata/classification.config
reference-config-file: /etc/suricata/reference.config

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify

```

```

Jan 08 17:34:48 talha-VMware-Virtual-Platform suricata[12894]: W: unix-manager:
root@talha-VMware-Virtual-Platform:/home/talha# ls /var/lib/suricata/rules
ls: cannot access '/var/lib/suricata/rules': No such file or directory
root@talha-VMware-Virtual-Platform:/home/talha# nano /etc/suricata/suricata.yaml
root@talha-VMware-Virtual-Platform:/home/talha# sudo nano /var/lib/suricata/rules/suricata.rules
root@talha-VMware-Virtual-Platform:/home/talha# sudo suricata-update
8/1/2026 17:39:03 - <Info> - Using data-directory /var/lib/suricata

```

Alerts setup for Sql injection in local rules files

```

Type here to search
My Computer
  Metasploitable2-Linux
  Ubuntu 64-bit
  talha kali

root@talha-VMware-Virtual-Platform:/home/talha
GNU nano 7.2 /var/lib/suricata/rules/local.rules *
alert http $EXTERNAL_NET any -> $HOME_NET 80 (
  msg:"SQL Injection Attempt - OR 1=1";
  flow:to_server,established;
  content:"or";
  nocase;
  content:"1=1";
  nocase;
  http_uri;
  sid:1000010;
  rev:1;
)
alert http $EXTERNAL_NET any -> $HOME_NET 80 (
  msg:"SQL Injection Attempt - UNION SELECT";
  flow:to_server,established;
  content:"union";
  nocase;
  content:"select";
  nocase;
  http_uri;
  sid:1000011;
)

```

Now setup yaml files for attack

Check yaml files to loaded to configuration to cache

Cmd `sudo suricata -T -c /etc/suricata/suricata.yaml`

```
s/local.rules
root@talha-VMware-Virtual-Platform:/home/talha# sudo suricata -T -c /etc/suricata/suricata.yaml
Info: conf-yaml-loader: Configuration node 'EXTERNAL_NET' redefined.
i: suricata: This is Suricata version 8.0.2 RELEASE running in SYSTEM mode
i: mpm-hs: Rule group caching - loaded: 0 newly cached: 113 total cacheable: 113
i: suricata: Configuration provided was successfully loaded. Exiting.
root@talha-VMware-Virtual-Platform:/home/talha#
```

6. Open Metasploitable

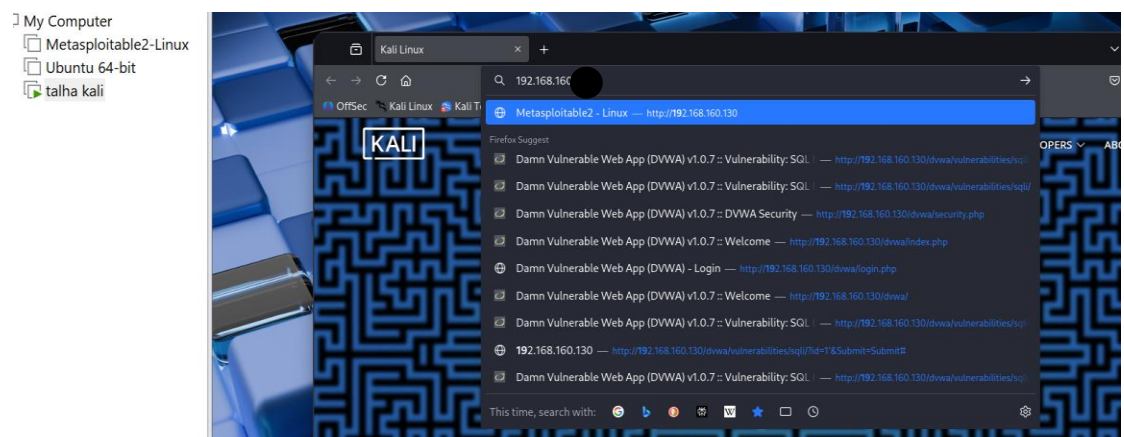
```
The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

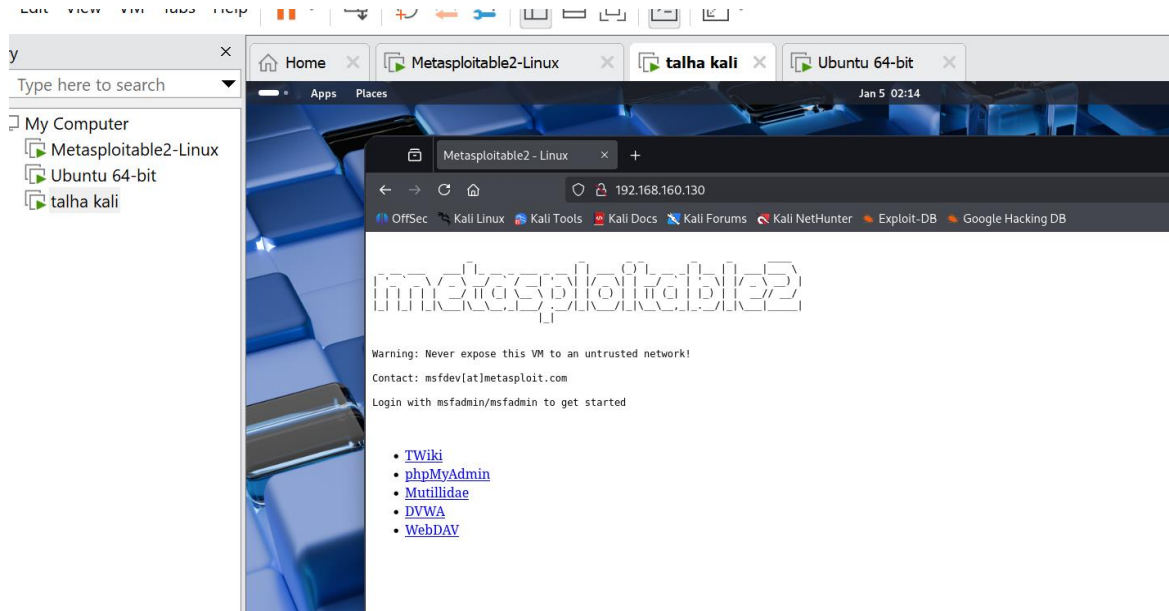
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 00:0c:29:7d:e4:b7 brd ff:ff:ff:ff:ff:ff
    inet 192.168.160.24/24 brd 192.168.160.255 scope global eth0
        inet6 fe80::20c:29:7d:e4:b7/64 scope link
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST> mtu 1500 qdisc noop qlen 1000
    link/ether 00:0c:29:7d:e4:c1 brd ff:ff:ff:ff:ff:ff
msfadmin@metasploitable:~$
```

7. Startup kali linux

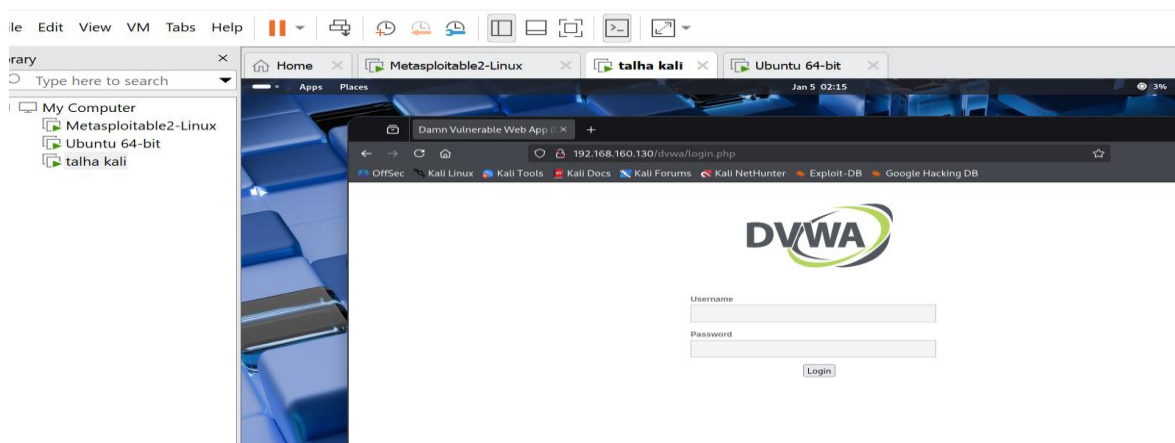
- Open Firefox
- Enter ip of metasploitable in Search
- Open metasploitable 2

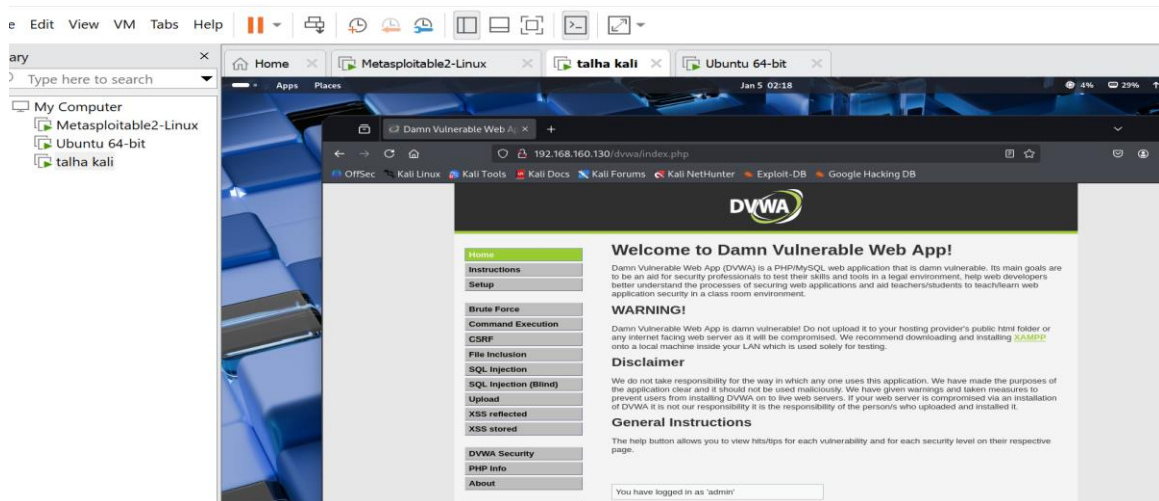




8. Open DVWA

- Open DVWA
- Make account
- Enter to low Security
- Select Sql injection

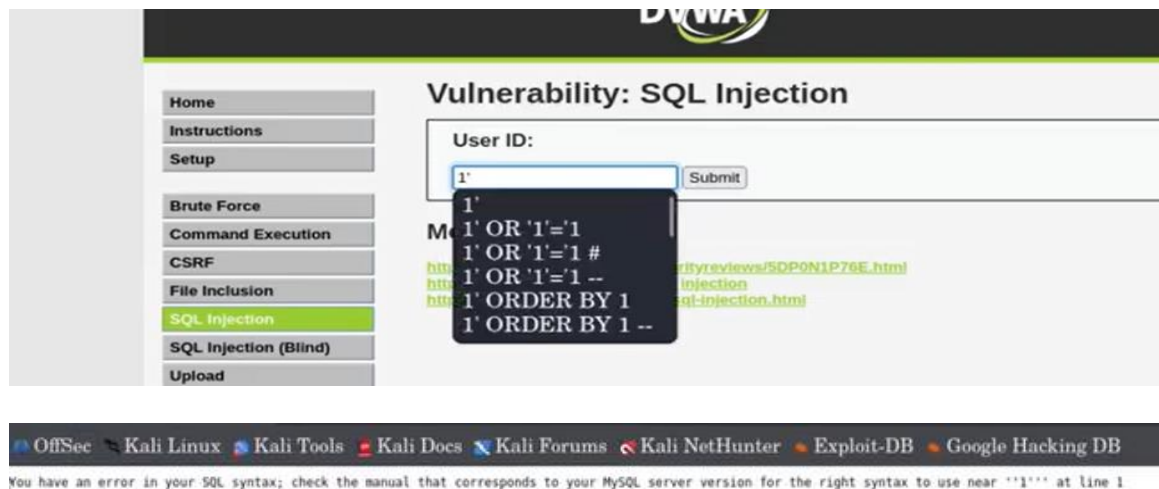




9. Sql Injection commands

Cmd 1'

If the application does not properly sanitize input, this causes a **SQL syntax error**, which confirms that the application may be vulnerable to SQL Injection



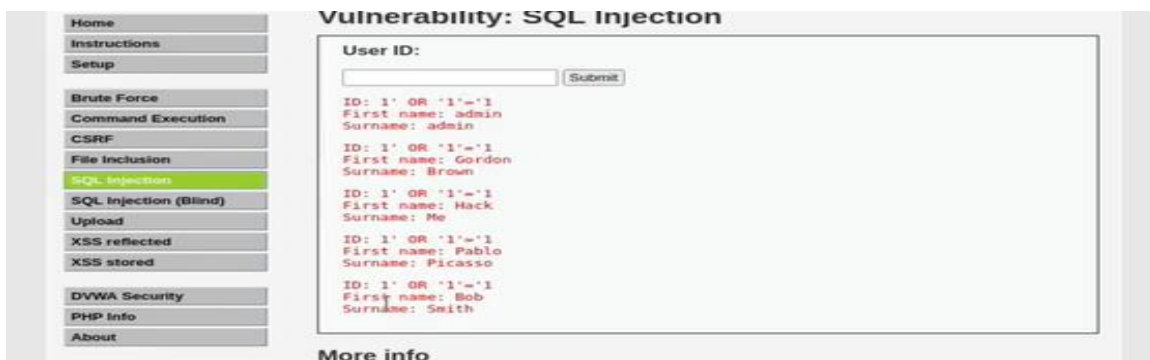
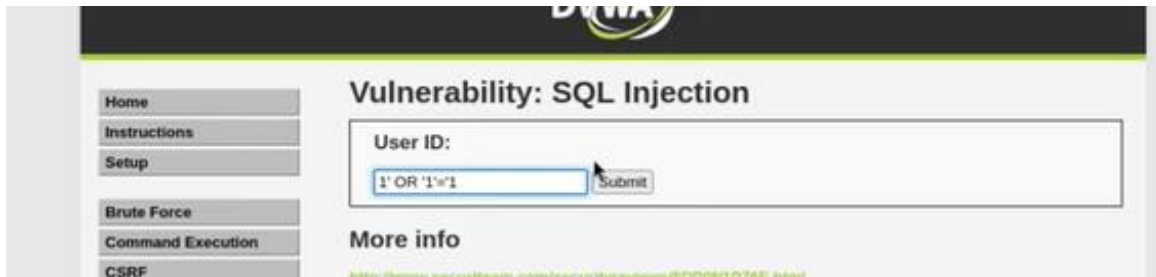
CMD 1' OR '1'='1

To bypass authentication

To retrieve all rows from a database table

Commonly used in login bypass attacks

Suricata identifies the use of SQL logical operators (OR) combined with tautologies (1=1), which are strong indicators of SQL Injection



Cmd 1' UNION SELECT 1, @@version#

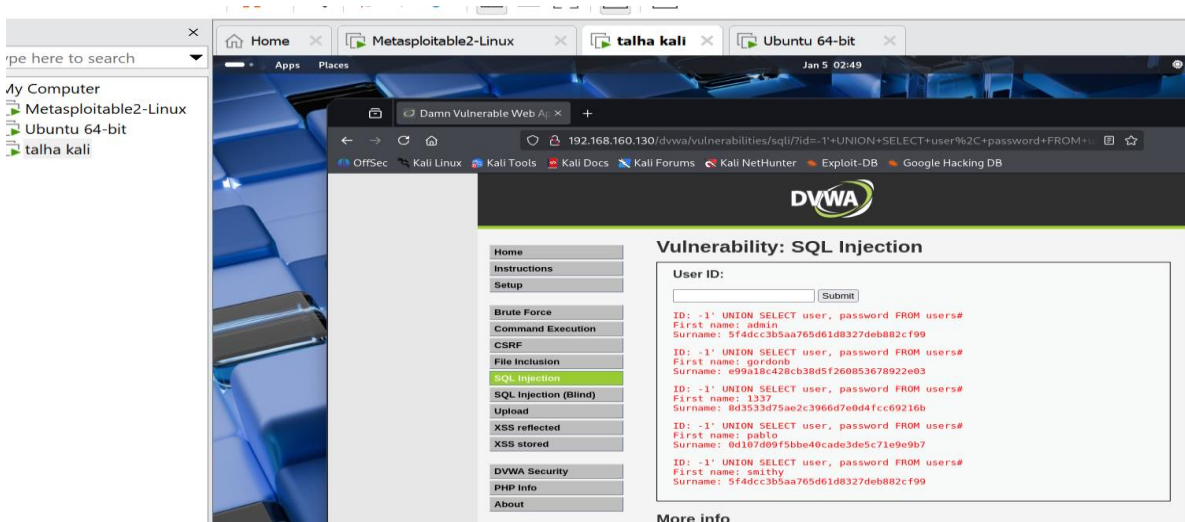
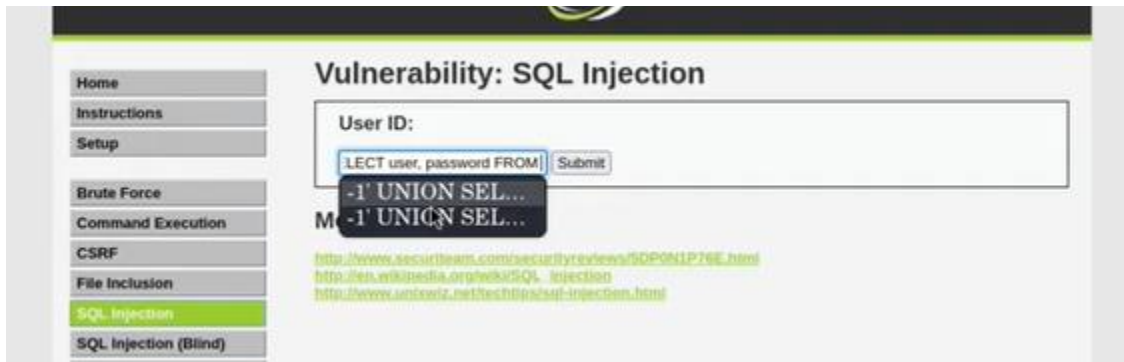
This payload uses the UNION SELECT statement to combine results from another query. @@version retrieves the database version, and # comments out the rest of the original SQL query



Cmd -1' UNION SELECT user, password FROM users#

This payload attempts to extract sensitive data from the database. It selects user and password fields from the users table using UNION SELECT.

The -1 ensures the original query returns no results, so injected results are displayed instead



E. Logs & Evidence

- fast.log: Shows SQL Injection alert summary
- Cmd sudo tail -f /var/log/suricata/fast.log
Sudo tail -f /var/log/suricata/eve.json

```
01/08/2026-18:39:46.053897 [**] [1:2010963:7] ET WEB_SERVER SELECT USER SQL Injection Attempt in URI [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 192.168.160.139:52144 -> 192.168.160.130:80
01/08/2026-18:39:46.053897 [**] [1:1000011:1] SQL Injection Attempt - UNION SELECT [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.160.139:52144 -> 192.168.160.130:80
01/08/2026-18:39:46.053897 [**] [1:1000012:1] SQL Injection Attempt - SQL Comment [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.160.139:52144 -> 192.168.160.130:80
01/08/2026-18:40:44.364269 [**] [1:2010963:7] ET WEB_SERVER SELECT USER SQL Injection Attempt in URI [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 192.168.160.139:55608 -> 192.168.160.130:80
01/08/2026-18:40:44.364269 [**] [1:1000011:1] SQL Injection Attempt - UNION SELECT [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.160.139:55608 -> 192.168.160.130:80
01/08/2026-18:40:44.364269 [**] [1:1000012:1] SQL Injection Attempt - SQL Comment [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.160.139:55608 -> 192.168.160.130:80
```

```
talha@talha-VMware-Virtual-Platform:~$ sudo tail -f /var/log/suricata/fast.log
[01/08/2026-18:37:23.909724 [**] [1:2011037:6] ET WEB_SERVER Possible Attempt to Get SQL Server Version in URI using SELECT VERSION [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 192.168.160.139:37446 -> 192.168.160.130:80
01/08/2026-18:37:23.909724 [**] [1:2014890:3] ET WEB_SERVER Possible attempt to enumerate MS SQL Server version [**] [Classification: Attempted Administrator Privilege Gain] [Priority: 1] {TCP} 192.168.160.139:37446 -> 192.168.160.130:80
01/08/2026-18:37:23.909724 [**] [1:1000011:1] SQL Injection Attempt - UNION SELECT [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.160.139:37446 -> 192.168.160.130:80
01/08/2026-18:37:23.909724 [**] [1:1000012:1] SQL Injection Attempt - SQL Comment [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.160.139:37446 -> 192.168.160.130:80
01/08/2026-18:39:46.053897 [**] [1:2010963:7] ET WEB_SERVER SELECT USER SQL Injection Attempt in URI [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 192.168.160.139:52144 -> 192.168.160.130:80
01/08/2026-18:39:46.053897 [**] [1:1000011:1] SQL Injection Attempt - UNION SELECT [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.160.139:52144 -> 192.168.160.130:80
01/08/2026-18:39:46.053897 [**] [1:1000012:1] SQL Injection Attempt - SQL Comment [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.160.139:52144 -> 192.168.160.130:80
01/08/2026-18:40:44.364269 [**] [1:2010963:7] ET WEB_SERVER SELECT USER SQL Injection Attempt in URI [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 192.168.160.139:55608 -> 192.168.160.130:80
01/08/2026-18:40:44.364269 [**] [1:1000011:1] SQL Injection Attempt - UNION SELECT [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.160.139:55608 -> 192.168.160.130:80
01/08/2026-18:40:44.364269 [**] [1:1000012:1] SQL Injection Attempt - SQL Comment [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.160.139:55608 -> 192.168.160.130:80
```

F. Findings

Suricata successfully detected SQL Injection attempts.
Alerts triggered due to SQL keywords in HTTP traffic.
Minor false positives possible.

G. Conclusion

This project successfully demonstrated how a SQL Injection attack can be detected using the Suricata Intrusion Detection System in a controlled virtual environment. By using Kali Linux as the attacker, Metasploitable 2 with DVWA as the victim, and Ubuntu running Suricata as the detection system, a realistic and practical lab setup was created. The SQL Injection attacks generated through DVWA produced malicious HTTP traffic, which was effectively captured and analyzed by Suricata.

During the experiment, Suricata generated alerts in both `fast.log` and `eve.json` files, clearly identifying SQL Injection patterns such as the use of single quotes, logical operators, and `UNION SELECT` statements. This shows that Suricata is capable of detecting common web application attacks by analyzing network traffic and matching it against predefined rules. The project also helped in understanding how alerts are structured, including signature IDs (SID), alert messages, and severity levels.

Overall, this project provided hands-on experience with intrusion detection, log analysis, and web application security. It highlighted the importance of deploying IDS tools like Suricata to monitor network traffic and detect malicious activities in real time. This practical exercise strengthened the understanding of how SQL Injection attacks work and how they can be detected before causing serious damage to systems and data.